МИНИСТЕРСТВО ОБРАЗОВАНИЯ
Брестский государственный технический университет
Кафедра ИИТ

ОТЧЕТ

по лабораторной работе №4

«Архитектура и программирование сопроцессора. Использование целочисленных и вещественных команд»

Выполнила:
студентка 4-го курса
группы АС-66
Демидович А.А.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных команд.

1. Постановка задачи

1. Изучить теоретические сведения: архитектура и программная модель сопроцессора – регистры и их назначение; форматы чисел, типовые команды, целочисленные команды.
2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды согласно варианту, выданному преподавателем (вариант 2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществить через консоль.
3. Протестировать программу по всем условиям ветвления и подготовиться к защите.

2. Вариант задания

Вариант 2. Вычислить значение кусочно-заданной функции:
y = 4xy + 5, при x + y > 10;
y = (3xy + 2y) / (x^2 + y^2 + 1), при x + y < -2;
y = 6x - 2y^2 + 1, при -2 <= x + y <= 1.

3. Текст программы (целочисленный вариант)

```
.686
.model flat, c
option casemap:none

includelib D:\masm32\lib\msvcrt.lib
includelib D:\masm32\lib\kernel32.lib

printf PROTO C :ptr sbyte, :vararg
scanf  PROTO C :ptr sbyte, :vararg
system PROTO C :ptr sbyte
exit   PROTO C :dword

.data
    msg_x       db "Enter integer X: ", 0
    msg_y       db "Enter integer Y: ", 0
    fmt_in      db "%d", 0
    fmt_res     db "Result Z = %d", 10, 0
    fmt_undef   db "Undefined for 1 < x + y <= 10", 10, 0
    cmd_pause   db "pause", 0

    val_x       dd 0
    val_y       dd 0
    res_z       dd 0
    sum_xy      dd 0

    c_1         dd 1
```

```
    c_2         dd 2
    c_3         dd 3
    c_4         dd 4
    c_5         dd 5
    c_6         dd 6

.code

; --- Ветка 1: 4*x*y + 5 ---
Branch1_Int PROC
    fild dword ptr [val_x]
    fimul dword ptr [val_y]
    fimul dword ptr [c_4]
    fiadd dword ptr [c_5]
    fistp dword ptr [res_z]
    ret
Branch1_Int ENDP

; --- Ветка 2: (3xy + 2y) / (x^2 + y^2 + 1) ---
Branch2_Int PROC
    fild dword ptr [val_x]
    fimul dword ptr [val_y]
    fimul dword ptr [c_3]       ; st(0) = 3xy

    fild dword ptr [val_y]
    fimul dword ptr [c_2]       ; st(0) = 2y, st(1) = 3xy
    faddp st(1), st(0)          ; st(0) = 3xy + 2y

    fild dword ptr [val_x]
    fimul dword ptr [val_x]     ; st(0) = x^2, st(1) = числитель

    fild dword ptr [val_y]
    fimul dword ptr [val_y]     ; st(0) = y^2, st(1) = x^2, st(2) = числитель
    faddp st(1), st(0)          ; st(0) = x^2 + y^2, st(1) = числитель
    fiadd dword ptr [c_1]       ; st(0) = x^2 + y^2 + 1

    fdivp st(1), st(0)          ; st(0) = числитель / знаменатель
    fistp dword ptr [res_z]
    ret
Branch2_Int ENDP

; --- Ветка 3: 6*x - 2*(y^2) + 1 ---
Branch3_Int PROC
    fild dword ptr [val_x]
    fimul dword ptr [c_6]       ; st(0) = 6x

    fild dword ptr [val_y]
    fimul dword ptr [val_y]
    fimul dword ptr [c_2]       ; st(0) = 2*(y^2), st(1) = 6x

    fsubp st(1), st(0)          ; st(0) = 6x - 2y^2
    fiadd dword ptr [c_1]       ; st(0) = 6x - 2y^2 + 1
    fistp dword ptr [res_z]
    ret
Branch3_Int ENDP

; --- Основная функция вычисления ---
Calculate_Int PROC
    finit

    mov eax, [val_x]
    add eax, [val_y]
    mov [sum_xy], eax
```

```
    cmp eax, 10
    jg  do_b1

    cmp eax, -2
    jl  do_b2

    cmp eax, 1
    jle do_b3

    invoke printf, offset fmt_undef
    ret

do_b1:
    call Branch1_Int
    jmp print_res
do_b2:
    call Branch2_Int
    jmp print_res
do_b3:
    call Branch3_Int

print_res:
    invoke printf, offset fmt_res, [res_z]
    ret
Calculate_Int ENDP

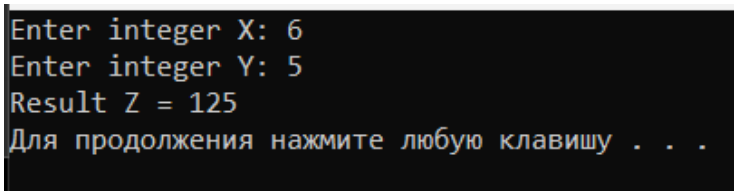
main PROC
    invoke printf, offset msg_x
    invoke scanf, offset fmt_in, offset val_x
    invoke printf, offset msg_y
    invoke scanf, offset fmt_in, offset val_y

    call Calculate_Int

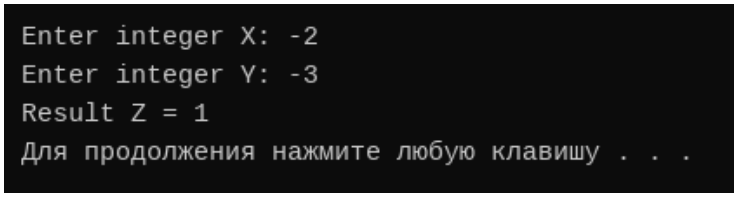
    invoke system, offset cmd_pause
    invoke exit, 0
main ENDP
END main
```

4. Скриншоты результатов (Раздел 1)

Тест 1 (Ветка 1: x + y > 10, ввод X = 6, Y = 5, результат Z = 125):



Тест 2 (Ветка 2: x + y < -2, ввод X = -2, Y = -3, результат Z = 1):



Тест 3 (Ветка 3: -2 <= x + y <= 1, ввод X = 1, Y = -1, результат Z = 5):

```
Enter integer X: 1
Enter integer Y: -1
Result Z = 5
Для продолжения нажмите любую клавишу . . .
```

Тест 4 (Вне диапазона: 1 < x + y <= 10, ввод X = 2, Y = 2):

```
Enter integer X: 2
Enter integer Y: 2
Undefined for 1 < x + y <= 10
Для продолжения нажмите любую клавишу . . .
```

Лабораторная работа №1. Раздел 2. Архитектура и программирование сопроцессора. Использование вещественных команд

Цель работы: изучение архитектуры и программной модели сопроцессора; практическая работа с системой вещественных команд.

1. Постановка задачи

1. Изучить систему вещественных команд сопроцессора (FLD, FSTP, FMUL, FDIV, FADD, FSUB, FCOMIP и др.).
2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды согласно варианту, выданному преподавателем (вариант 2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществить через консоль.
3. Протестировать программу по всем условиям ветвления и подготовиться к защите.

2. Вариант задания

Вариант 2. Вычислить значение кусочно-заданной функции над вещественными числами одинарной точности (REAL4):
y = 4xy + 5, при x + y > 10;
y = (3xy + 2y) / (x^2 + y^2 + 1), при x + y < -2;
y = 6x - 2y^2 + 1, при -2 <= x + y <= 1.

3. Текст программы (вещественный вариант)

```
.686
.model flat, c
option casemap:none

includelib D:\masm32\lib\msvcrt.lib
includelib D:\masm32\lib\kernel32.lib
```

```
printf PROTO C :ptr sbyte, :vararg
scanf  PROTO C :ptr sbyte, :vararg
system PROTO C :ptr sbyte
exit   PROTO C :dword

.data
    msg_x       db "Enter real X: ", 0
    msg_y       db "Enter real Y: ", 0
    fmt_in      db "%f", 0
    fmt_res     db "Result Z = %.4f", 10, 0
    fmt_undef   db "Undefined for 1 < x + y <= 10", 10, 0
    cmd_pause   db "pause", 0

    val_x       real4 0.0
    val_y       real4 0.0
    res_z       real8 0.0

    c_1         real4 1.0
    c_2         real4 2.0
    c_3         real4 3.0
    c_4         real4 4.0
    c_5         real4 5.0
    c_6         real4 6.0
    c_10        real4 10.0
    c_minus2    real4 -2.0

.code

; --- Ветка 1: 4*x*y + 5 ---
Branch1_Real PROC
    fld dword ptr [val_x]
    fmul dword ptr [val_y]
    fmul dword ptr [c_4]
    fadd dword ptr [c_5]
    fstp qword ptr [res_z]
    ret
Branch1_Real ENDP

; --- Ветка 2: (3xy + 2y) / (x^2 + y^2 + 1) ---
Branch2_Real PROC
    fld dword ptr [val_x]
    fmul dword ptr [val_y]
    fmul dword ptr [c_3]       ; st(0) = 3xy

    fld dword ptr [val_y]
    fmul dword ptr [c_2]       ; st(0) = 2y, st(1) = 3xy
    faddp st(1), st(0)          ; st(0) = 3xy + 2y

    fld dword ptr [val_x]
    fmul st(0), st(0)          ; st(0) = x^2, st(1) = числитель

    fld dword ptr [val_y]
    fmul st(0), st(0)          ; st(0) = y^2, st(1) = x^2, st(2) = числитель
    faddp st(1), st(0)          ; st(0) = x^2 + y^2
    fadd dword ptr [c_1]       ; st(0) = x^2 + y^2 + 1

    fdivp st(1), st(0)          ; st(0) = числитель / знаменатель
    fstp qword ptr [res_z]
    ret
Branch2_Real ENDP

; --- Ветка 3: 6*x - 2*(y^2) + 1 ---
```

```
Branch3_Real PROC
    fld dword ptr [val_x]
    fmul dword ptr [c_6]       ; st(0) = 6x

    fld dword ptr [val_y]
    fmul st(0), st(0)          ; y^2
    fmul dword ptr [c_2]       ; 2 * y^2

    fsubp st(1), st(0)          ; 6x - 2y^2
    fadd dword ptr [c_1]       ; 6x - 2y^2 + 1
    fstp qword ptr [res_z]
    ret
Branch3_Real ENDP

; --- Основная функция вычисления ---
Calculate_Real PROC
    finit

    fld dword ptr [val_x]
    fadd dword ptr [val_y]

    fld dword ptr [c_10]
    fcomip st(0), st(1)
    jb  do_b1

    fld dword ptr [c_minus2]
    fcomip st(0), st(1)
    ja  do_b2

    fld dword ptr [c_1]
    fcomip st(0), st(1)
    jae do_b3

    fstp st(0)
    invoke printf, offset fmt_undef
    ret

do_b1:
    fstp st(0)
    call Branch1_Real
    jmp print_res

do_b2:
    fstp st(0)
    call Branch2_Real
    jmp print_res

do_b3:
    fstp st(0)
    call Branch3_Real

print_res:
    invoke printf, offset fmt_res, dword ptr [res_z], dword ptr [res_z + 4]
    ret
Calculate_Real ENDP

main PROC
    invoke printf, offset msg_x
    invoke scanf, offset fmt_in, offset val_x
    invoke printf, offset msg_y
    invoke scanf, offset fmt_in, offset val_y

    call Calculate_Real
```

```
    invoke system, offset cmd_pause
    invoke exit, 0
main ENDP
END main
```

4. Скриншоты результатов (Раздел 2)

Тест 1 (Ветка 1: x + y > 10, ввод X = 6, Y = 5, результат Z = 125.0000):

```
Enter real X: 6
Enter real Y: 5
Result Z = 125.0000
Для продолжения нажмите любую клавишу . . .
```

Тест 2 (Ветка 2: x + y < -2, ввод X = -2, Y = -3, результат Z = 0.8571):

```
Enter real X: -2
Enter real Y: -3
Result Z = 0.8571
Для продолжения нажмите любую клавишу . . .
```

Тест 3 (Ветка 3: -2 <= x + y <= 1, ввод X = 1, Y = -1, результат Z = 5.0000):

```
Enter real X: 1
Enter real Y: -1
Result Z = 5.0000
Для продолжения нажмите любую клавишу . . .
```

Тест 4 (Вне диапазона: 1 < x + y <= 10, ввод X = 2, Y = 2):

```
Enter real X: 2
Enter real Y: 2
Undefined for 1 < x + y <= 10
Для продолжения нажмите любую клавишу . . .
```

Вывод

В ходе выполнения лабораторной работы были изучены архитектура и стек регистров сопроцессора (FPU), а также особенности программирования с использованием целочисленных и вещественных команд. В первом разделе были применены целочисленные команды (FILD, FIMUL, FIADD, FIDIV, FISTP), производящие преобразование целочисленных форматов при загрузке и округление при выгрузке (что при делении во второй ветке дало целочисленный результат 1). Во втором разделе были использованы вещественные команды (FLD, FMUL, FADD, FDIV, FSTP) и инструкция сравнения вещественных чисел FCOMIP, что обеспечило высокую точность вычислений с плавающей точкой (результат второй

ветки составил 0.8571). Все процедуры протестированы по всем заданным ветвям алгоритма, результаты совпали с расчетными значениями.